

Mid-Term BTP Presentation on

**“MILP Heuristics in 3D Bin Packing Problem”**

|                           |                             |
|---------------------------|-----------------------------|
| <b>BTP Supervisor(s):</b> | Prashant Palkar             |
| <b>Submitted by:</b>      | Nirvan Nagrale(2022ME12048) |
| <b>Project No:</b>        | 116                         |



DEPARTMENT OF MECHANICAL ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY, DELHI

September 2025

# Literature review and Research gap

- The **3D Bin Packing Problem (3D-BPP)** is a strongly NP-hard problem with wide applications in logistics, supply chain management, and packaging.  
It involves arranging rectangular SKUs into fixed-dimension bins while minimizing bin usage or maximizing volume utilization.
- **Exact approaches** (MILP formulations) give optimal solutions for small/medium instances but scale poorly for large datasets due to combinatorial explosion.
- **Heuristic methods** address this by trading off accuracy for speed:
  - *Layer-Based*: creates horizontal layers of identical boxes and stacks them.
  - *Peak Filling Slice Push*: identifies “peaks” left by tall stacks and fills them with smaller items.
  - *Space Defragmentation*: repositions items to merge fragmented gaps and reinserts leftovers.
- **Research Gap**: Lack of scalable methods that robustly integrate heuristics and MILP for large, heterogeneous, multi-DFC data; limited exploration of decomposition and column generation.



# Problem Statement/ BTP Objectives

- The 3D-BPP requires efficient packing with orientation, weight balance, and bin constraints.
- MILP solvers (e.g., CPLEX) scale poorly, while heuristics give inconsistent results.
- A hybrid approach can combine heuristic feasibility with MILP optimization.
- The main objective is to **minimize unused volume**, expressed as:

$$\min \left( \sum_{j \in J} \sum_{k \in K_j} u_{jk} L_j W_j H_j - \sum_{i \in I} l_i w_i h_i \right)$$

## Objectives

- Develop and implement Layer-Based, Peak Filling, and Space Defragmentation heuristics.
- Integrate heuristics with MILP (CPLEX) to balance scalability and solution quality.
- Evaluate performance on real picklists: volumetric efficiency, time efficiency and objective function value.
- Identify bottlenecks and propose improvements like delayed column generation and decomposition techniques.



- **Tools & Platforms**

- Python (numpy, pandas, itertools, dataclasses) for heuristic algorithms.
- IBM ILOG CPLEX for MILP optimization.
- High-performance Atreyee server accessed through PuTTY for execution.

- **Framework**

The project employs a **hybrid heuristic–MILP approach**:

- Heuristics rapidly generate feasible packings using rules like layer stacking, gap filling, and compaction.
  - These solutions are used as **initial inputs for MILP**
  - The combination aims to reduce solver runtime while still achieving near-optimal efficiency.
- This balances the **trade-off**: MILP provides accuracy but poor scalability; heuristics provide scalability but weaker solutions.



- **Layer-Based Heuristic**

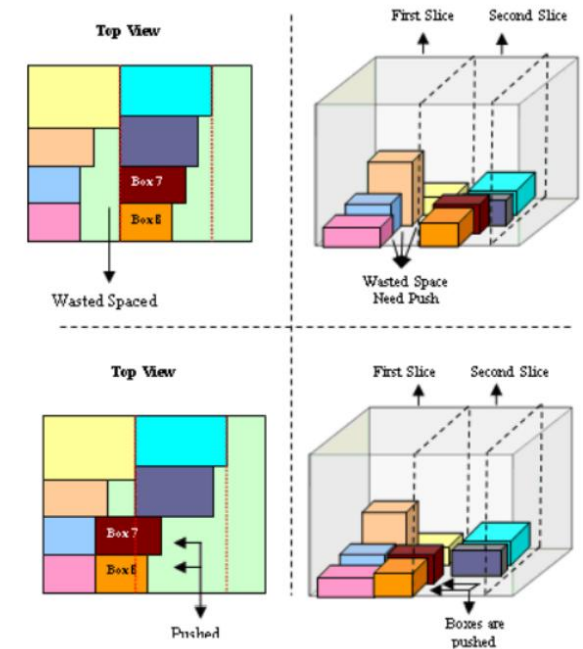
- Generates all possible horizontal layers by item type and orientation.
- Combines these layers vertically into candidate solutions.
- Selects final combinations balancing weight and volume.
- *Issue:* exponential growth in combinations → runtime blowup.

- **Peak Filling Slice Push Heuristic**

- Places items into slices, creating tall peaks.
- Identifies gaps beside peaks and fills them with smaller items.
- Repeats iteratively until no gaps can be filled.
- *Advantage:* fast; *Limitation:* Generates some solutions with 0% efficiency.

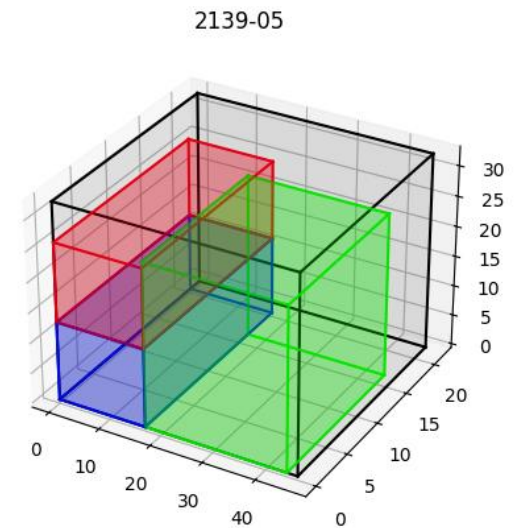
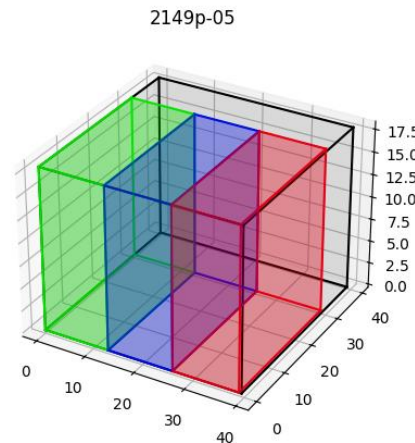
- **Space Defragmentation Heuristic**

- Initial insertion using extreme points.
- Detects fragmented gaps and shifts items along axes to consolidate space.
- Attempts to reinsert unpacked items in the new free regions.
- *Benefit:* higher density; *Drawback:* heavy computation and debugging



- **Implementation**

- Modular Python code: separate files for each heuristic, MILP interface, data handling, visualization.
- Visualization: matplotlib 3D plots of bins with wireframes and coloured cuboids.
- Data: outputs stored in [Excel](#) (efficiency %, runtimes, solver flags).
- Excel file also shows graphical representation of comparison between heuristics.



- **Hybrid MILP + Heuristics**

- Main CPLEX MILP model alone often matched or outperformed heuristic-initialized runs.
- For complex picklists, MILP reached time limits even with heuristic inputs.
- Some heuristics provided near-zero efficiency due to orientation and gap-handling bugs.

## Challenges Observed

- *Layer-Based*: exponential runtime, impractical for large SKU sets.
- *Peak Filling*: dependency on quality of initial placement; easily got stuck.
- *Space Defragmentation*: high overhead and unstable improvements.
- Debugging complexity: SKU orientation logic, bin indexing, and iterative repositioning consumed significant development time.



- Built a **modular hybrid heuristic–MILP framework** for solving the 3D-BPP.
- Heuristics offered faster feasible solutions but failed to consistently improve MILP performance.
- MILP baseline remained competitive, often outperforming heuristic-assisted runs.
- Major issues: **runtime explosion** (Layer-Based), **limited improvements** (Peak Filling), **overhead** (Defragmentation).
- Results match trends in literature: heuristics rarely outperform exact MILP for real-world picklists.

## Future Directions:

- Refine heuristics to provide stronger initial solutions.
- Explore decomposition and benchmarking against state-of-the-art methods.



